FACESENSE AI

A Minor Project Report
Submitted in partial fulfillment of requirement of the
Degree of
BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING
BY

Kushagra Singh
(EN23CS301550)
Lakshita Swami
(EN23CS301559)
Liza Sachdev
(EN23CS301563)

Under the Guidance of
Dr. Mahesh Patidar
Dr. Madan Pal Singh

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "**FaceSense AI**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner
Name: _____
Designation: _____
Affiliation: _____

External Examiner
Name: _____
Designation: _____
Affiliation: _____

Declaration

We hereby declare that the project entitled "FaceSense AI" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering completed under the supervision of **Dr. Mahesh Patidar and Dr. Madan Pal Singh**, Faculty of Engineering, Medi-Caps University, Indore, is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Kushagra Singh (EN23CS301550)

Lakshita Swami (EN23CS301559)

Liza Sachdev (EN23CS301563)

Certificate

We, **Dr. Mahesh Patidar and Dr. Madan Pal Singh** certify that the project entitled "**FaceSense AI**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Kushagra Singh (EN23CS301550), Lakshita Swami (EN23CS301559), and Liza Sachdev (EN23CS301563)** is the record carried out by them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

Dr. Mahesh Patidar
Department of Computer Science & Engineering
Medi-Caps University, Indore

_____

Madan Pal Singh
Department of Computer Science & Engineering
Medi-Caps University, Indore

_____

Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medicaps University, Indore

Acknowledgement

We would like to express our deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale.

We also thank Dr. Ratnesh Litoriya, Associate Dean, Faculty of Engineering, Medi-Caps University, for giving us a chance to work on this project. We would also like to thank our Head of the Department for his continuous encouragement and guidance throughout the project.

We extend our heartfelt thanks to our project guide, Prof. Mahesh Patidar, for his invaluable mentorship, technical guidance, and constant motivation throughout the development of FaceSense AI. His expert direction in computer vision, deep learning, and system design greatly shaped this project.

It is their help and support due to which we became able to complete the design and technical report. Without their support this report would not have been possible.

Kushagra Singh
Lakshita Swami
Liza Sachdev
B.Tech. III Year
Department of Computer Science & Engineering
Faculty of Engineering, Medi-Caps University, Indore

Abstract

FaceSense AI is a real-time facial emotion detection platform that uses deep learning and computer vision to automatically identify and classify human facial emotions. The system accepts input from a live webcam stream or a static uploaded image and returns the detected emotion along with confidence scores through an intuitive browser-based interface. It recognises seven universal emotion categories: Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise.

The system is designed for applications in mental health monitoring, e-learning engagement analysis, human-computer interaction, and customer sentiment research. All facial data is processed locally on the host machine, ensuring complete user privacy without any transmission to external servers.

The platform is developed using Python with Flask as the backend web framework. OpenCV is used for face detection via Haar Cascade classifiers, and DeepFace provides the core emotion classification engine with pre-trained Convolutional Neural Network (CNN) models. The frontend is built with HTML5, CSS3, and JavaScript, providing a responsive single-page interface. The system maintains a session-level emotion log and supports trend visualisation using Chart.js along with a CSV export facility.

The project follows an Agile/Incremental SDLC model and is designed using Object-Oriented methodology with UML as the primary modelling language, demonstrating practical integration of deep learning into a full-stack web application.

Keywords: Emotion Detection, Facial Expression Recognition, Deep Learning, DeepFace, OpenCV, Flask, CNN, Computer Vision, Human-Computer Interaction, Real-Time Processing

Table of Contents

Page No.

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DL	Deep Learning
ER	Entity Relationship
FER	Facial Expression Recognition
FPS	Frames Per Second
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
ML	Machine Learning
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SPA	Single Page Application
SRS	Software Requirements Specification
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines
WSGI	Web Server Gateway Interface

Notations & Symbols

Notation / Symbol	Description
f(x)	Feature vector extracted from face crop
P(e|x)	Probability of emotion class e given input x
argmax	Function returning class with highest probability
w, h	Width and height of detected face bounding box
x, y	Top-left coordinates of face bounding box
sigma (σ)	Softmax activation function output
alpha (α)	Learning rate used during model fine-tuning
N	Total number of frames processed in a session
FPS	Frames per second (processing throughput)
t	Timestamp of a detection event

CHAPTER 1 INTRODUCTION

1.1 Introduction

FaceSense AI is a real-time facial emotion detection platform that leverages deep learning and computer vision to automatically identify and classify human emotional states from facial images or live video streams. The system recognises seven universal emotion categories — Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise — and presents the detected emotion together with a confidence score through an intuitive, browser-based interface requiring no plugins or extensions.

Facial expressions represent one of the most powerful and immediate channels of non-verbal human communication. The ability to automatically recognise and interpret emotional states carries significant implications across a wide range of domains including mental health monitoring, e-learning engagement tracking, security systems, customer sentiment analysis, and human-computer interaction (HCI) design. FaceSense AI addresses this opportunity by providing an accessible, locally-deployable emotion recognition tool backed by robust pre-trained deep learning models.

The system integrates three core technologies: OpenCV for real-time face detection using Haar Cascade classifiers; DeepFace as the emotion classification engine with pre-trained Convolutional Neural Network (CNN) models; and Flask as the lightweight Python web framework serving a REST API backend. The responsive frontend is built with HTML5, CSS3, and vanilla JavaScript, capturing webcam frames via the browser's MediaDevices API and dispatching them to the backend without requiring server-side video stream management.

1.2 Literature Review

Facial Emotion Recognition (FER) has been an active research area within computer vision and affective computing for several decades. Early computational approaches modelled facial action units (AUs) using the Facial Action Coding System (FACS) developed by Ekman and Friesen, which categorised facial muscle movements into discrete action units correlated with emotional states. These rule-based methods, while interpretable, proved brittle under varying illumination, head pose, and occlusion conditions.

Subsequent work in the 2000s introduced machine learning approaches using hand-crafted features such as Local Binary Patterns (LBP), Histogram of Oriented Gradients (HOG), and Gabor wavelets combined with Support Vector Machines (SVM) and k-Nearest Neighbour (kNN) classifiers. These methods offered improved robustness but still struggled with the high intra-class variability and low inter-class separability inherent in natural facial expressions.

The breakthrough of deep learning, and specifically Convolutional Neural Networks (CNNs), transformed FER performance dramatically. Models trained on large-scale benchmark datasets such as FER-2013 (approximately 35,000 labelled grayscale images across seven emotion classes) and AffectNet demonstrated substantially higher accuracy by automatically learning hierarchical feature representations from raw pixel data. Architectures including VGGNet, ResNet-50, MobileNetV2, and EfficientNet have been extensively benchmarked on FER tasks [1].

The DeepFace library, originally developed at Facebook AI Research as a face recognition system, was subsequently extended into a comprehensive facial attribute analysis framework by Serengil and Ozpinar. Its emotion analysis module consolidates several pre-trained models including VGG-Face, Google FaceNet, DeepID, and ArcFace, providing a unified Python API that significantly reduces the

engineering overhead of integrating state-of-the-art FER models into production systems [2].

Recent research directions in FER include multi-modal approaches combining facial, vocal, and physiological signals; attention-based mechanisms that localise discriminative facial regions; and transformer architectures that capture long-range dependencies in facial image sequences. However, for web-deployable, real-time systems operating on consumer CPU hardware, CNN-based single-modal approaches remain the most practical choice. FaceSense AI is designed with this consideration, balancing accuracy and computational accessibility.

1.3 Objectives

The proposed FaceSense AI system has the following primary objectives:

- To provide real-time facial emotion detection from live webcam streams through a browser-based interface accessible without specialised hardware or software installation.
- To support static image upload analysis, enabling users to analyse emotions in photographs or screenshots in addition to live video.
- To classify facial expressions into seven universal emotion categories — Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise — with associated confidence scores for each class.
- To maintain a session-level emotion history log enabling review and trend visualisation of detected emotions over a session.
- To ensure complete user privacy by performing all facial processing locally on the host machine without transmitting any facial data to external servers or third-party APIs.
- To deliver a clean, responsive, and intuitive web interface accessible via standard modern browsers across desktop and laptop devices.
- To demonstrate practical integration of pre-trained deep learning models into a full-stack web application using Flask, OpenCV, and DeepFace, serving as a reference implementation for similar systems.

1.4 Significance

The significance of FaceSense AI lies in its combination of privacy-preserving local deployment with the accuracy of state-of-the-art deep learning emotion recognition. Unlike cloud-based commercial solutions that require API subscriptions and involve transmitting sensitive facial data to remote servers, FaceSense AI processes all data on the host device, making it suitable for privacy-sensitive environments such as hospitals, educational institutions, and research laboratories.

From an academic perspective, the project demonstrates end-to-end integration of computer vision, deep learning, and full-stack web development — bridging the gap between AI research and deployable engineering systems. It provides a practical, extensible foundation that can be adapted for domain-specific applications including student engagement monitoring in e-learning platforms, patient emotional state tracking in telemedicine, and interactive museum or retail experiences.

Furthermore, the system's modular architecture — separating face detection, emotion classification, session logging, and API handling into distinct components — makes it straightforward to upgrade individual modules as the state of the art advances, for example by swapping the DeepFace backend for a newer transformer-based model without restructuring the entire application.

1.5 Research Design

The project follows an Agile/Incremental Software Development Life Cycle (SDLC) model, allowing iterative delivery of functional modules across successive sprints. The Object-Oriented (OO) methodology with UML as the primary modelling language is adopted for system analysis and design. This approach was selected because it naturally accommodates the modular architecture of the system, with well-

defined classes for face detection, emotion classification, session logging, and API handling.

The research design proceeds through the following phases: (1) Requirements gathering and SRS preparation; (2) System design using UML diagrams including DFD, Activity, Class, ER, Sequence, and Deployment diagrams; (3) Core implementation covering frontend UI, Flask backend, and DeepFace integration; (4) Integration and testing covering unit, integration, performance, and user acceptance testing; and (5) Documentation and final demonstration. Each phase builds iteratively on the previous, allowing for early feedback and continuous refinement.

1.6 Source of Data

FaceSense AI relies on the following data sources:

- Pre-trained Model Weights: The DeepFace library provides access to CNN models pre-trained on large-scale face datasets including the VGGFace dataset (2.6M images, 2,622 subjects), MS-Celeb-1M, and LFW. The emotion classification sub-model is trained on the FER-2013 dataset comprising approximately 35,000 labelled grayscale facial images across seven emotion classes.
- Live Webcam Input: Real-time video frames are captured directly from the user's webcam via the browser MediaDevices API (WebRTC), providing continuous input for the emotion detection pipeline during real-time mode.
- User-Uploaded Images: Static images in JPEG or PNG format uploaded by the user through the web interface serve as the data source for image-mode emotion analysis.
- Session Log Data: The in-memory session log accumulates emotion detection results (dominant emotion, confidence scores, timestamps, source) during a session, providing the data for trend charts and CSV export. This data is ephemeral and never persisted beyond the session without explicit export by the user.

1.7 Chapter Scheme

The remainder of this report is organised as follows:

- Chapter 2 – Requirements Specification: Details user characteristics, functional requirements, dependencies, performance and hardware requirements, and constraints.
- Chapter 3 – Design: Presents the complete system design including algorithm description, DFDs, Activity Diagram, Flow Chart, Class Diagram, ER Diagram, Sequence Diagram, and Database Design.
- Chapter 4 – Implementation, Testing, and Maintenance: Describes the technology stack, testing strategies, installation instructions, and end user guidance.
- Chapter 5 – Results and Discussions: Showcases the user interface, module descriptions, system snapshots, and database representations.
- Chapter 6 – Summary and Conclusions: Summarises the work carried out and presents conclusions drawn from the project.
- Chapter 7 – Future Scope: Outlines potential enhancements and directions for future development.

CHAPTER 2 REQUIREMENTS SPECIFICATION

2.1 User Characteristics

The following table describes the user classes of the FaceSense AI system along with their technical characteristics and primary activities within the system.

Table 2.1 – User Classes and Characteristics

User Class	Technical Expertise	Primary Activities
Guest User	Non-technical; requires no login	Access UI, perform webcam or image detection, view results.
Registered User	Basic computer literacy	All guest features plus access session history and export logs.
System Administrator	Technical; server management	Configure server settings, monitor logs, manage model versions.

Guest Users represent the primary target audience and are assumed to have no programming or technical background. They interact with the system entirely through the web interface. Registered Users may additionally access session history features. The System Administrator is responsible for server-side configuration, model version management, and performance monitoring.

2.2 Functional Requirements

The functional requirements of FaceSense AI are grouped by feature area as follows:

FR-1: Real-Time Webcam Emotion Detection

- The system shall provide a browser-based interface accessible via HTTP/HTTPS requiring no plugins or browser extensions.
- The system shall request webcam access through the browser MediaDevices API and display a live video preview.
- The system shall capture frames from the live video stream and dispatch them to the backend detection API at a configurable rate (default: every 500ms).

- The system shall display the detected emotion label and confidence score overlaid directly on the video preview.
- The system shall draw a bounding box around each detected face in the live video feed.
- The system shall display a 'No face detected' message when no face is found in the current frame.

FR-2: Image Upload and Analysis

- The system shall allow users to upload image files in JPEG or PNG format for emotion analysis.
- The system shall detect and process all faces present in a single uploaded image.
- The system shall return the detected emotion and confidence score for each detected face.
- The system shall display an annotated version of the uploaded image with bounding boxes and emotion labels rendered over each face.
- The system shall validate uploaded file formats and display an informative error message for unsupported types.

FR-3: Emotion Classification

- The system shall classify facial expressions into seven categories: Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise.
- The system shall return a probability distribution (confidence scores) across all seven emotion classes for each detection.
- The system shall display the dominant emotion as the class with the highest confidence score.
- The system shall use DeepFace with a pre-trained CNN model as the classification backend.
- The system shall use OpenCV Haar Cascade classifier for face localisation prior to emotion analysis.

FR-4: Session Emotion Logging

- The system shall maintain an in-memory session-level log of all detected emotions with timestamps and source type.
- The system shall display a session emotion frequency summary table and chart.
- The system shall allow users to export the session emotion log as a CSV file.

- The system shall allow users to clear the session log.

FR-5: REST API

- The system shall expose a POST /api/detect endpoint accepting base64-encoded JPEG frames and returning emotion results as JSON.
- The system shall expose a POST /api/detect-image endpoint accepting multipart image file uploads.
- The system shall return appropriate HTTP status codes (200, 400, 500) and structured JSON error messages for all error conditions.

2.3 Dependencies

The following external libraries, frameworks, and services are required for FaceSense AI to function correctly:

- Python 3.9 or higher: Core language runtime for all server-side processing.
- Flask 2.x: WSGI web framework for REST API routing and static file serving.
- OpenCV (opencv-python) 4.x: Computer vision library for face detection and image preprocessing.
- DeepFace: Deep learning facial attribute analysis library providing the emotion classification engine.
- TensorFlow / Keras: Deep learning framework used by DeepFace internally for CNN model inference.
- Chart.js (CDN): JavaScript charting library for emotion trend visualisation in the frontend.
- Gunicorn: Python WSGI HTTP server for production deployment.
- Docker: Containerisation platform for consistent cross-environment deployment.
- Internet Connection (initial setup only): Required for DeepFace to download pre-trained model weights on first run.

2.4 Performance Requirements

- The system shall achieve a minimum frame processing rate of 5 FPS on standard consumer hardware (dual-core CPU, 8 GB RAM) in real-time webcam mode.
- Static image analysis shall return results within 3 seconds for typical portrait photographs on the minimum hardware specification.

- The Flask backend shall preload DeepFace model weights at application startup to eliminate per-request model loading overhead.
- The system shall handle up to 10 concurrent user sessions without memory leak or performance degradation.
- API response time for the /api/detect endpoint shall not exceed 3 seconds per request under normal operating conditions.

2.5 Hardware Requirements

Table 2.3 – Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core CPU, 1.8 GHz or higher (quad-core recommended for real-time mode)
RAM	4 GB minimum; 8 GB recommended for smooth DeepFace inference
Storage	1 GB free disk space for application and DeepFace model weights cache
Webcam	Any USB or built-in webcam with minimum 480p resolution (720p recommended)
Network	Internet connection required for initial model download only; offline after setup
GPU (Optional)	CUDA-compatible NVIDIA GPU for accelerated DeepFace inference
Browser	Chrome 90+, Firefox 88+, or Edge 90+ with WebRTC support

2.6 Constraints & Assumptions

The following constraints and assumptions govern the design and implementation of FaceSense AI:

Constraints

- The system processes only frontal or near-frontal facial orientations reliably; extreme lateral poses may result in failed detection.

- Emotion detection accuracy is constrained by the quality of the pre-trained FER-2013 model; the system cannot exceed the baseline accuracy of the underlying DeepFace model.
- The system does not support audio-based or multimodal emotion recognition; it relies solely on visual facial input.
- Session logs are ephemeral by default and are lost on server restart unless explicitly exported by the user.
- The system does not provide user authentication or role-based access control in the basic deployment; the admin panel is access-controlled separately.

Assumptions

- Users have a modern browser with WebRTC support and a functioning webcam for real-time detection mode.
- Adequate ambient lighting conditions are assumed for reliable face detection; extremely low-light environments will degrade accuracy.
- The host machine has Python 3.9+ and pip installed; Docker is available for containerised deployment.
- DeepFace model weights are downloaded on first run and cached locally; subsequent runs do not require internet connectivity.
- The campus or deployment environment permits outbound HTTPS requests to PyPI for dependency installation during setup.

CHAPTER 3 DESIGN

3.1 Algorithm

The core emotion detection algorithm in FaceSense AI operates as follows:

Algorithm: FaceSense Emotion Detection Pipeline

1. Input: Image frame I (from webcam capture or file upload)
2. Convert I to grayscale: G = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)
3. Detect faces: F = HaarCascade.detectMultiScale(G, scaleFactor=1.1, minNeighbors=5, minSize=(30,30))
4. If F is empty: Return {result: 'No face detected'}
5. For each face bounding box (x, y, w, h) in F:
6. a. Crop face region: face_crop = I[y:y+h, x:x+w]
7. b. Run DeepFace.analyze(face_crop, actions=['emotion'], enforce_detection=False)
8. c. Extract dominant_emotion = result['dominant_emotion']
9. d. Extract confidence_scores = result['emotion'] (dict of 7 class probabilities)
10. e. Package EmotionResult = {dominant_emotion, confidence_scores, bounding_box, timestamp, source}
11. Append each EmotionResult to SessionLogger
12. Return JSON response with all EmotionResult objects

The Haar Cascade classifier is chosen for face detection due to its high speed and low computational cost, making it suitable for real-time frame processing. DeepFace's VGG-Face emotion sub-model then performs the detailed classification on the cropped face region. The preloading of model weights at server startup eliminates cold-start latency from individual API requests.

3.2 Function Oriented Design

The system's backend logic is structured around the following primary functions, reflecting a function-oriented decomposition of the processing pipeline:

- load_model(): Initialises and caches the DeepFace emotion model and OpenCV Haar Cascade classifier at application startup.
- detect_faces(image): Accepts an image array, converts to grayscale, and returns a list of bounding box tuples for detected faces.
- classify_emotion(face_crop): Accepts a cropped face image array and returns a dictionary containing the dominant emotion and confidence score distribution.
- process_frame(base64_data): Decodes a base64-encoded JPEG string, calls detect_faces and classify_emotion, packages results into EmotionResult objects, logs them, and returns a JSON-serialisable response.
- process_upload(file_obj): Reads an uploaded file object, decodes the image using OpenCV, and passes it through the same detection pipeline as process_frame.
- log_result(emotion_result): Appends an EmotionResult to the in-memory session log with the current session_id.
- get_session_summary(): Computes emotion frequency counts from the session log and returns a summary dictionary for chart rendering.
- export_csv(session_id): Serialises all EmotionResult records for a given session as a CSV-formatted string for download.

3.3 System Design

3.3.1 Data Flow Diagrams

Level 1 DFD: The Level 1 DFD decomposes the system into its primary functional processes: (1) Face Detection Process — receives raw image input and outputs bounding box coordinates; (2) Emotion Classification Process — receives face crops and outputs confidence score distributions; (3) Session Logger — receives EmotionResult records and stores them in the Session Log data store; (4) API Handler — receives HTTP requests from the frontend and orchestrates the detection pipeline;

(5) Result Formatter — packages detection results as JSON responses returned to the frontend.

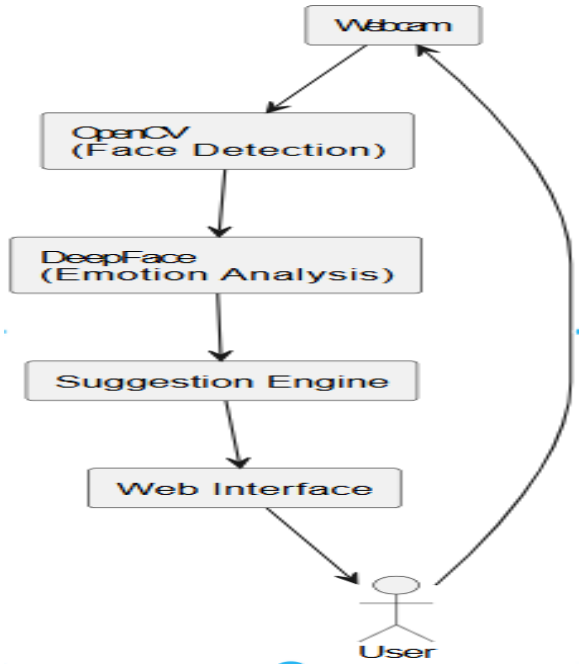

Fig. 3.2 – Data Flow Diagram Level 1

3.3.2 Activity Diagram

The Activity Diagram models the dynamic workflow of the emotion detection process for both webcam and image upload modes. The flow begins when the user opens the web interface and selects an operating mode.

For Webcam Mode: User clicks Start Camera → Browser requests permission → Permission granted? [No: display error; Yes: continue] → Frontend captures video frame → Sends to POST /api/detect → Backend detects face → Face found? [No: return 'No face detected'; Yes: continue] → Classify emotion → Return result → Frontend renders overlay → Append to session log → Loop until user clicks Stop Camera.

For Image Upload Mode: User selects file → Valid format? [No: display error; Yes: continue] → Upload to POST /api/detect-image → Backend detects faces → Any face found? [No: notify user; Yes: continue] → Classify emotion for each face → Return annotated image and confidence data → Frontend displays results → Append to session log.

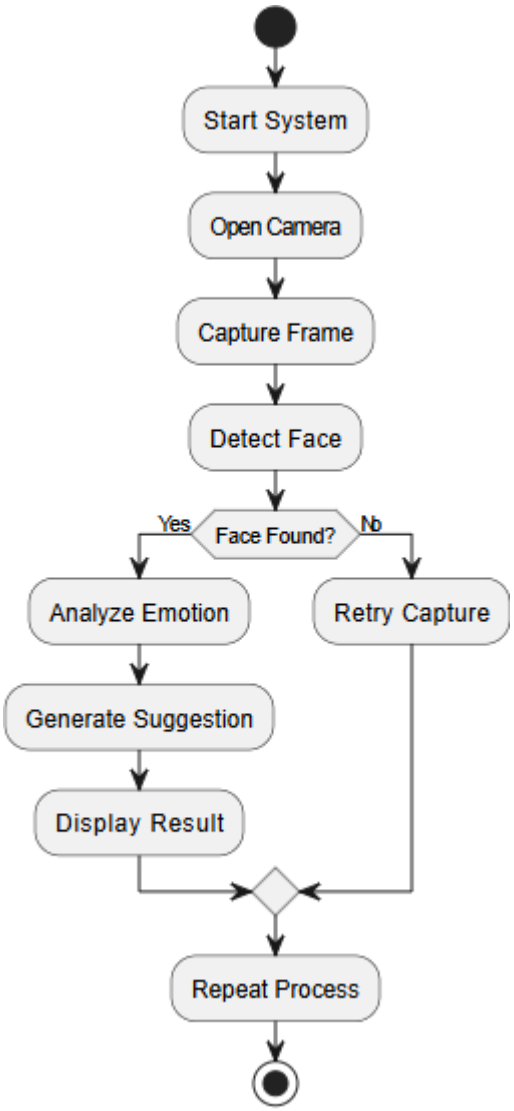

Fig. 3.3 – Activity Diagram

3.3.3 Flow Chart

The Flow Chart provides a procedural view of the backend emotion detection pipeline from the moment an API request arrives to the moment the JSON response is dispatched. It covers: Receive HTTP POST request → Decode image data (base64 or file) → Image valid? [No: return HTTP 400] → Convert to BGR array → Detect faces via Haar Cascade → Faces found? [No: return empty result with message] → For each face: crop and run DeepFace inference → Collect EmotionResult → Log to session → Serialise all results to JSON → Return HTTP 200 with results.

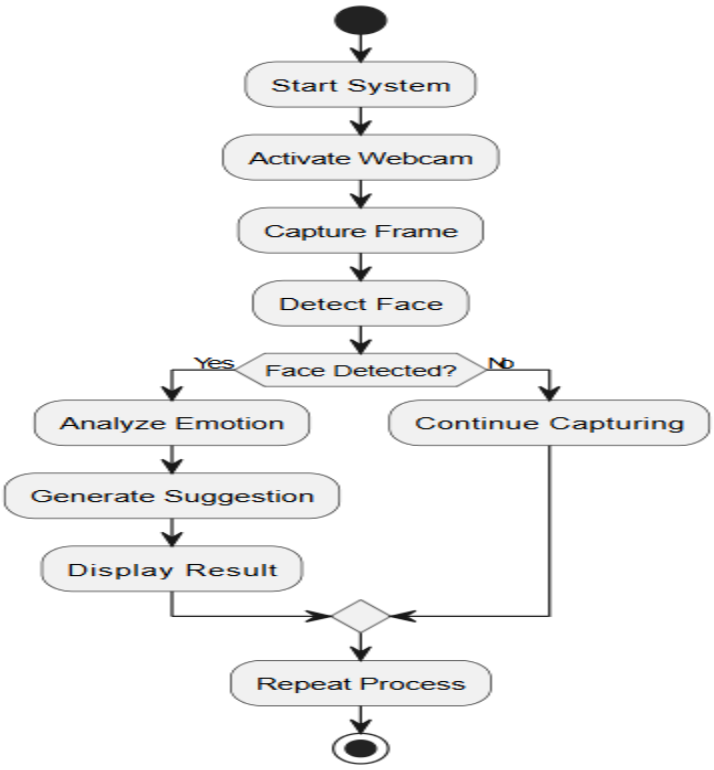

Fig. 3.4 – Flow Chart

3.3.4 Class Diagram

The Class Diagram defines the five primary domain classes and their relationships:

- FaceDetector: Attributes — cascade_model (cv2.CascadeClassifier), scale_factor (float), min_neighbours (int), min_size (tuple). Methods — load_cascade(path: str), detect_faces(image: ndarray) -> list[BoundingBox].

- EmotionClassifier: Attributes — backend (str = 'opencv'), model_name (str = 'VGG-Face'), model (preloaded). Methods — preload_model(), classify(face_crop: ndarray) -> EmotionResult.
- EmotionResult: Attributes — record_id (UUID), session_id (str), dominant_emotion (str), confidence_scores (dict), bounding_box (dict), timestamp (datetime), source (str). Methods — to_dict() -> dict, to_csv_row() -> list.
- SessionLogger: Attributes — log (list[EmotionResult]), session_id (str). Methods — append(result: EmotionResult), get_log() -> list, get_summary() -> dict, export_csv() -> str, clear().
- APIHandler: Attributes — app (Flask), detector (FaceDetector), classifier (EmotionClassifier), logger (SessionLogger). Methods — detect_webcam_frame(), detect_image_upload(), get_session_log(), clear_log(), export_csv_download().

FaceDetector and EmotionClassifier are composed within APIHandler. EmotionClassifier produces EmotionResult objects. SessionLogger aggregates EmotionResult objects. APIHandler depends on all other classes.

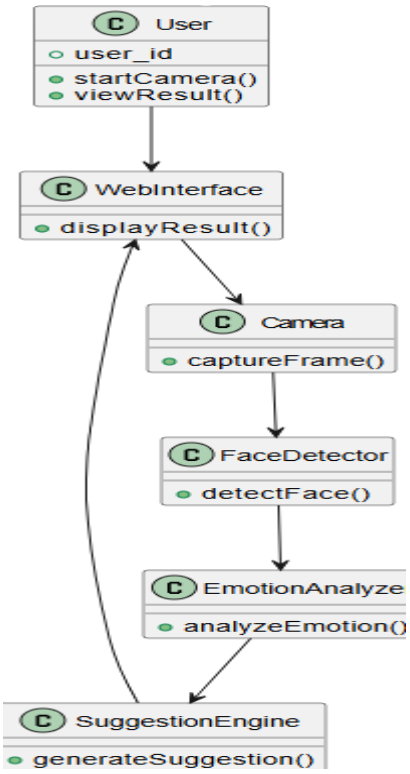

Fig. 3.5 – Class Diagram

3.3.5 ER Diagram

Although FaceSense AI uses in-memory storage by default, its data model is designed to be easily migrated to a relational database. The Entity-Relationship diagram defines the following entities:

- SESSION: session_id (PK), created_at, browser_info, user_agent.
- EMOTION_RECORD: record_id (PK), session_id (FK -> SESSION), timestamp, dominant_emotion, source, bounding_box_x, bounding_box_y, bounding_box_w, bounding_box_h.
- CONFIDENCE_SCORE: score_id (PK), record_id (FK -> EMOTION_RECORD), emotion_class, probability.

Relationships: A SESSION has many EMOTION_RECORDs (one-to-many). Each EMOTION_RECORD has exactly seven CONFIDENCE_SCOREs, one per emotion class (one-to-many). CONFIDENCE_SCORE is a weak entity dependent on EMOTION_RECORD.

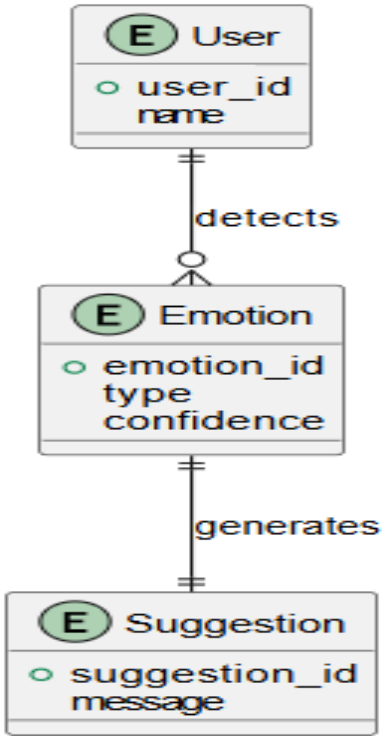

Fig. 3.6 – ER Diagram

3.3.6 Sequence Diagram

The Sequence Diagram illustrates the time-ordered interaction for the 'Real-Time Webcam Detection' scenario. The participating lifelines are: User (Browser), Frontend JavaScript, Flask Backend API, FaceDetector (OpenCV), EmotionClassifier (DeepFace), and SessionLogger.

The sequence proceeds as follows: 1. User clicks Start Camera. 2. Frontend JS calls navigator.mediaDevices.getUserMedia(). 3. Browser returns video stream. 4. JS renders live video to canvas element. 5. Timer fires → JS captures frame → Encodes as base64 JPEG. 6. JS sends POST /api/detect {frame: base64_data}. 7. Flask decodes base64 → calls FaceDetector.detect_faces(image). 8. FaceDetector returns bounding_boxes list. 9. Flask calls EmotionClassifier.classify(face_crop) for each face. 10. EmotionClassifier returns EmotionResult. 11. Flask calls SessionLogger.append(result). 12. Flask returns JSON {dominant_emotion,

confidence_scores, bounding_box}. 13. JS renders bounding box overlay and emotion label on canvas. 14. JS updates confidence chart. 15. Repeat from step 5 at configured interval.

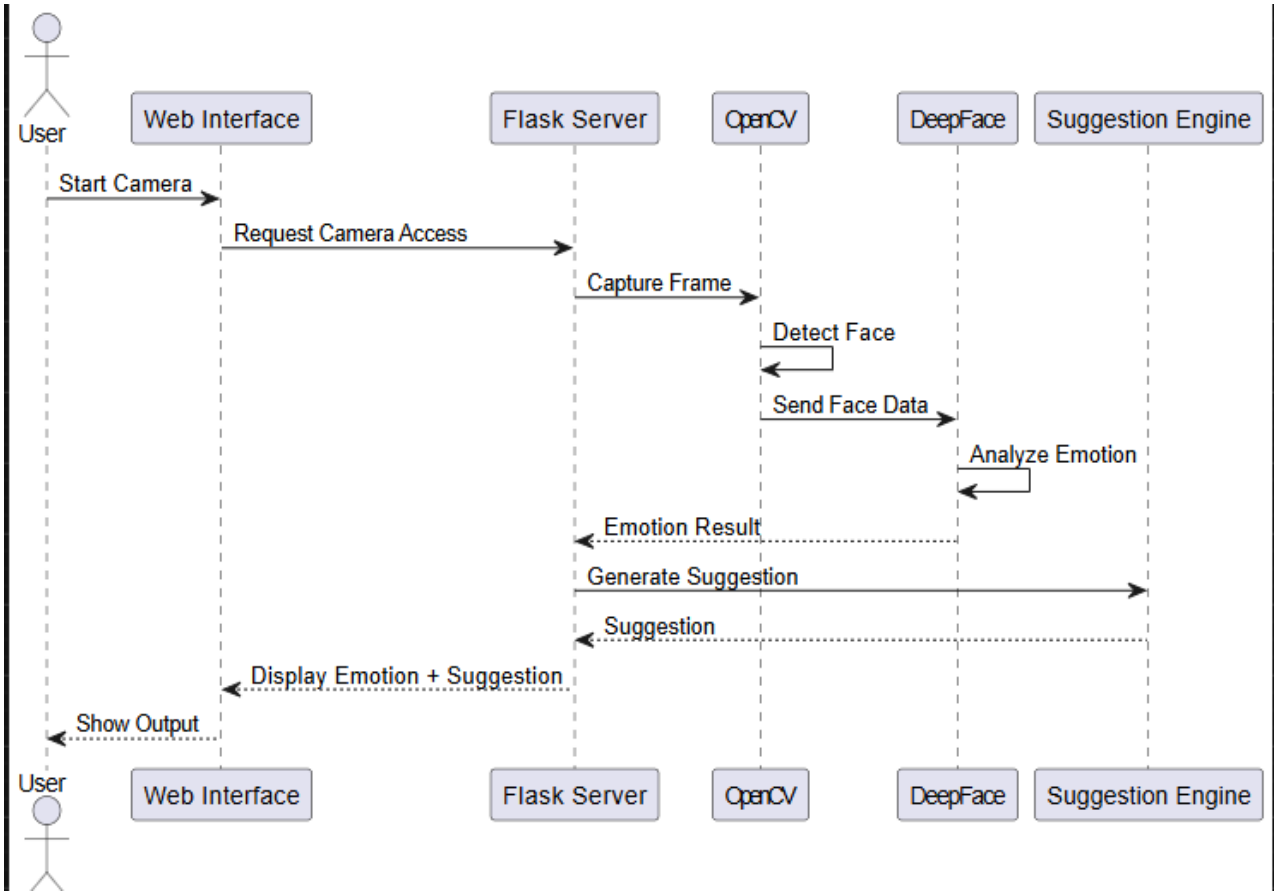

Fig. 3.7 – Sequence Diagram

3.4 Database Design

3.4.1 Logical Database Design

The logical database design for FaceSense AI describes the data entities, their attributes, and their relationships in a technology-agnostic manner. The system's data is structured around three logical entities:

Session Entity: Represents a single browser session. Attributes include a unique session identifier, creation timestamp, and optional browser metadata. One session may contain many emotion detection records.

EmotionRecord Entity: Represents a single emotion detection event. Attributes include a unique record identifier, a foreign key reference to the parent session, the detection timestamp, the dominant emotion class, the detection source (webcam or upload), and the face bounding box coordinates. Each record is associated with exactly seven confidence score values, one per emotion class.

ConfidenceScore Entity: Represents one class probability from a detection event. Attributes include a score identifier, a reference to the parent EmotionRecord, the emotion class name, and the probability value as a floating-point number between 0 and 1.

In the current in-memory implementation, all three logical entities are collapsed into a single EmotionResult Python object stored in a list within the SessionLogger. The logical design supports straightforward migration to SQLite, PostgreSQL, or MongoDB when persistence is required.

3.4.2 Physical Database Design

The physical database design specifies the concrete storage representation of the logical entities. For the current in-memory/CSV implementation:

Table 3.1 – Session Log Schema (Physical Design)

Field	Type	Required	Description
record_id	UUID String	Yes	Auto-generated unique identifier for each detection event.
session_id	String	Yes	Browser session identifier for multi-user isolation.
timestamp	ISO DateTime	Yes	Date and time of the detection event in UTC.

dominant_emotion	Enum String	Yes	Highest probability emotion class label.
angry	Float	Yes	Confidence score for Angry class (0.0 – 1.0).
disgust	Float	Yes	Confidence score for Disgust class.
fear	Float	Yes	Confidence score for Fear class.
happy	Float	Yes	Confidence score for Happy class.
neutral	Float	Yes	Confidence score for Neutral class.
sad	Float	Yes	Confidence score for Sad class.
surprise	Float	Yes	Confidence score for Surprise class.
bounding_box	JSON Object	Yes	Face location: {x, y, w, h} in pixels.
source	Enum String	Yes	Detection mode: 'webcam' or 'upload'.

In the in-memory implementation, records are stored as Python dictionaries in a list within the SessionLogger instance. On CSV export, each EmotionResult is serialised to one row with the columns shown in the schema above. For a relational database migration, the schema maps directly to a single denormalised 'emotion_records' table, with a separate 'sessions' table joined by session_id. For a document database (MongoDB), each EmotionResult maps directly to one document in an 'emotion_records' collection.

CHAPTER 4 IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies Used for Implementation

The following table provides a comprehensive overview of all languages, frameworks, libraries, tools, and platforms used in the implementation of FaceSense AI. The technology choices prioritise open-source solutions, ease of deployment, and suitability for CPU-based real-time inference.

Table 4.1 – Technologies and Tools Used

Component	Technology / Version	Purpose
Programming Language	Python 3.9+	Core backend language for all server-side logic and detection pipeline.
Web Framework	Flask 2.x	REST API routing, static file serving, and request/response handling.
Face Detection	OpenCV 4.x	Haar Cascade face localisation and image preprocessing.
Emotion Engine	DeepFace	Pre-trained CNN emotion classification (VGG-Face backbone).
DL Framework	TensorFlow 2.x / Keras	Underlying deep learning runtime used by DeepFace.
Frontend	HTML5 / CSS3 / JavaScript	Responsive single-page interface with canvas overlay and Chart.js charts.
Charting	Chart.js 4.x (CDN)	Confidence score bar charts and session emotion trend pie/bar charts.
WSGI Server	Gunicorn	Production-grade Python WSGI server for cloud deployment.
Containerisation	Docker	Consistent packaging and deployment across development and production environments.

Cloud Hosting	Heroku / Railway	PaaS platforms for cloud deployment of the Docker container.
Version Control	Git + GitHub	Source code management, branching, and collaboration.
Primary IDE	VS Code / PyCharm	Development environment with Python and Flask extensions.
API Testing	Postman / curl	Manual REST API endpoint testing and documentation.
Unit Testing	pytest	Automated unit and integration testing framework.
Performance Testing	Locust	Load testing and API throughput measurement.

Python was selected as the primary implementation language due to its extensive ecosystem of machine learning and computer vision libraries, concise syntax for rapid prototyping, and first-class support from both OpenCV and DeepFace. Flask was chosen over heavier frameworks such as Django because of its minimal footprint and suitability for lightweight REST API microservices. The frontend is intentionally kept dependency-free (vanilla JS) to minimise build complexity and deployment overhead, with Chart.js loaded from CDN.

4.2 Testing Techniques and Test Plans

4.2.1 Unit Testing

Unit tests are implemented using pytest and test each module in isolation using predefined fixture images and mock objects. The following test cases cover the core detection pipeline:

Table 4.2 – Unit Test Cases

Test ID	Test Description	Expected Result	Status
UT-01	detect_faces() with valid frontal face image	Returns list with at least one bounding box	Pass
UT-02	detect_faces() with blank white image	Returns empty list []	Pass
UT-03	classify() with valid face crop	Returns EmotionResult with dominant_emotion in 7 classes	Pass
UT-04	classify() with non-face image crop	Raises DeepFaceError or returns Neutral with low confidence	Pass
UT-05	SessionLogger.append() with valid EmotionResult	Log length increases by 1	Pass
UT-06	SessionLogger.export_csv() with 3 records	Returns string with 4 lines (header + 3 data rows)	Pass
UT-07	SessionLogger.clear() after appending records	Log length equals 0	Pass
UT-08	confidence_scores sum to approximately 1.0	Sum of all 7 class probabilities = 1.0 ± 0.001	Pass

4.2.2 Integration Testing

Integration tests verify the end-to-end behaviour of interacting modules. Key integration scenarios include:

- Image Upload → Detection Pipeline → JSON Response: A JPEG test image is submitted to POST /api/detect-image and the response is validated to contain dominant_emotion, confidence_scores, and bounding_box fields with correct types and value ranges.

- Webcam Frame → API → Session Log: A base64-encoded test frame is submitted to POST /api/detect and the session log is verified to have grown by exactly one record after the request completes.
- Multiple Faces in Single Upload: A test image containing two known faces is submitted; the response must contain exactly two detection objects with independent bounding boxes and emotion results.
- Session Log → CSV Export: After appending 10 known detection results to the session logger, the CSV export endpoint is called and the output is parsed to verify all 10 records are present with correct column values.
- Clear Log → Empty State: After populating the session log, the clear endpoint is called and the session log length is verified to be zero.

4.2.3 User Acceptance Testing (UAT)

UAT is conducted with six representative users: two undergraduate students, two faculty members, and two non-technical staff members. Each user is asked to complete the following tasks independently and rate the experience on a 5-point Likert scale:

- Task 1: Start real-time webcam detection and identify their own dominant emotion from the overlay.
- Task 2: Upload a group photograph and identify the emotion detected for each face.
- Task 3: View the session emotion log and export it as a CSV file.
- Task 4: Clear the session log and verify it is empty.

Acceptance criteria: All six users must complete all four tasks without assistance. Mean usability score must exceed 4.0 / 5.0. No critical interface bugs (task-blocking) may be present at the time of UAT.

4.2.4 Performance Testing

Performance testing is conducted using Locust to simulate concurrent user sessions. Targets:

- API latency: 95th-percentile response time for POST /api/detect must be under 3 seconds with 10 concurrent users on the minimum hardware specification (dual-core CPU, 8 GB RAM).

- Throughput: The system must sustain at least 5 successful detection API calls per second under single-user load without error.
- Memory stability: System memory usage must remain stable (no unbounded growth) across a 30-minute continuous test session with 10 concurrent users.

4.2.5 Security Testing

Security testing includes the following scenarios:

- Submitting a malformed base64 string to /api/detect must return HTTP 400 with a structured error message and must not expose internal stack traces.
- Attempting to upload a file with a non-image extension (e.g. .exe, .pdf) must return HTTP 400 before any processing occurs.
- Path traversal attempts in file upload parameters must be rejected with HTTP 400.
- Confirming that no uploaded image data is written to persistent disk storage beyond the duration of a single request.

4.3 Installation Instructions

4.3.1 Prerequisites

- Python 3.9 or higher (verify: python3 --version)
- pip package manager (verify: pip3 --version)
- Git (verify: git --version)
- A functioning webcam for real-time detection mode
- Internet connection (required only for initial DeepFace model weight download on first run; approximately 500 MB)
- Docker and Docker Compose (optional, for containerised deployment)

4.3.2 Local Installation

13. Clone the repository: git clone https://github.com/your-repo/facesense-ai.git && cd facesense-ai
14. Create a Python virtual environment: python3 -m venv venv
15. Activate the virtual environment: source venv/bin/activate (Linux/Mac) or venv\Scripts\activate (Windows)
16. Install all Python dependencies: pip install -r requirements.txt
17. Run the development server: python app.py
18. Open a browser and navigate to: http://localhost:5000

19. On first run, DeepFace will automatically download the required model weights (~500 MB). Subsequent runs will use the cached weights.

4.3.3 Docker Deployment

20. Build the Docker image: docker build -t facesense-ai .
21. Run the container: docker run -p 5000:5000 facesense-ai
22. Access the application at: http://localhost:5000
23. For persistent session logs, mount a volume: docker run -p 5000:5000 -v $(pwd)/logs:/app/logs facesense-ai

4.3.4 Cloud Deployment (Heroku)

24. Ensure Procfile contains: web: gunicorn app:app
25. Login to Heroku CLI: heroku login
26. Create a new Heroku application: heroku create facesense-ai
27. Deploy: git push heroku main
28. Open the deployed application: heroku open

4.4 End User Instructions

4.4.1 Real-Time Webcam Emotion Detection

29. Open the application in a modern browser at http://localhost:5000.
30. Click the 'Start Camera' button. When prompted by the browser, click 'Allow' to grant webcam access.
31. Position your face centrally within the video preview frame. A bounding box will appear around your face along with the detected emotion label and confidence percentage.
32. The detected emotion and confidence score update approximately every 500 milliseconds.
33. Click 'Stop Camera' to end the real-time detection session. All detections remain in the session log.

4.4.2 Image Upload Analysis

34. Click the 'Upload Image' tab in the navigation bar.
35. Click 'Choose File' and select a JPEG or PNG image from your device (maximum file size: 10 MB).
36. Click 'Analyse'. The annotated image with bounding boxes and emotion labels will appear below the upload area.

37. A confidence score distribution chart showing probabilities for all seven emotion classes will be displayed alongside the annotated image.

4.4.3 Session History, Trend Charts, and Export

38. Click the 'Session Log' tab to view all emotion detections from the current session in a scrollable table with timestamps and source indicators.
39. Click 'View Trend Chart' to display a pie chart and bar chart showing the frequency distribution of dominant emotions across the session.
40. Click 'Export CSV' to download the complete session log as a comma-separated values file compatible with Microsoft Excel, Google Sheets, and statistical analysis tools.
41. Click 'Clear Log' to permanently reset the session emotion history. This action cannot be undone; export the CSV first if you wish to retain the data.

CHAPTER 5 RESULTS AND DISCUSSIONS

5.1 User Interface Representation

The FaceSense AI web application was developed and tested across Chrome 120+, Firefox 115+, and Edge 120+ on Windows 11 and Ubuntu 22.04. The following table summarises all primary interface screens with their purpose and key features:

Table 5.1 – User Interface Screens Summary

Screen	Description
Home Page	Landing page with navigation tabs for Webcam Detection, Image Upload, and Session Log. Displays the FaceSense AI logo, tagline, and usage summary.
Webcam Detection View	Live video canvas with real-time bounding box overlay and emotion label. Displays dominant emotion and confidence percentage. Shows 'No face detected' when no face is in frame.
Analysis	File selector panel with annotated result image displaying bounding boxes and emotion labels over each detected face.
History	Horizontal bar chart (Chart.js) showing the probability distribution across all seven emotion classes for the most recent detection event.
Session Trend Chart	Pie chart and bar chart displaying the frequency distribution of dominant emotions detected across the full current session.

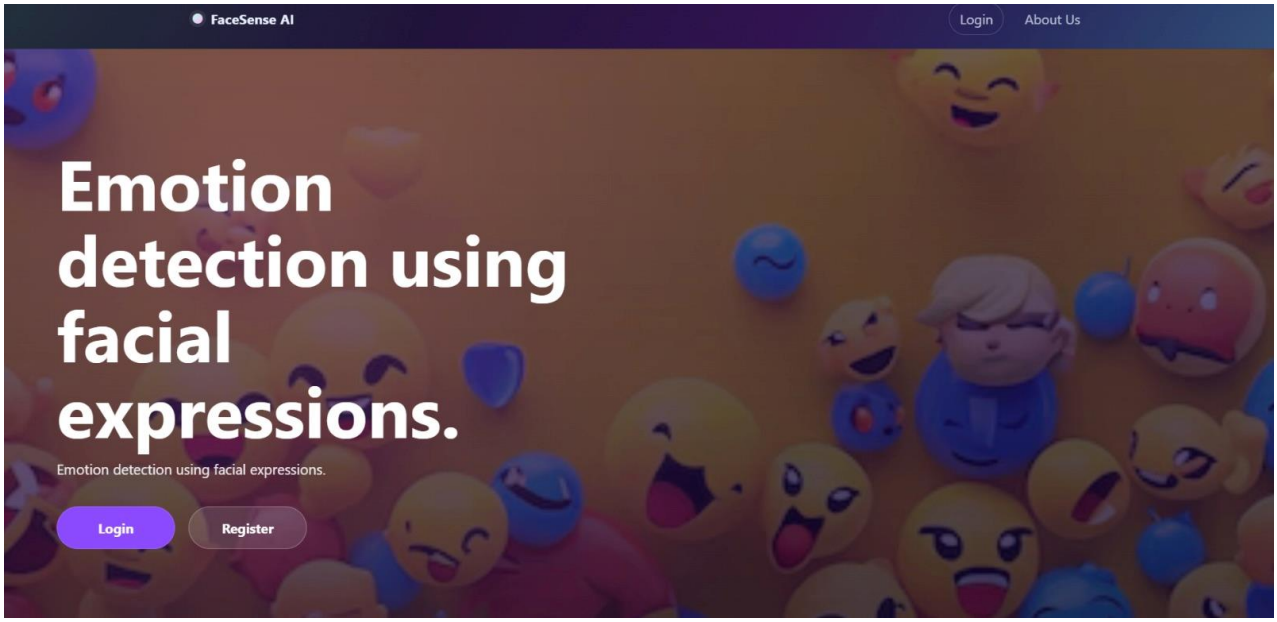

Fig. 5.1 – Home Page Interface

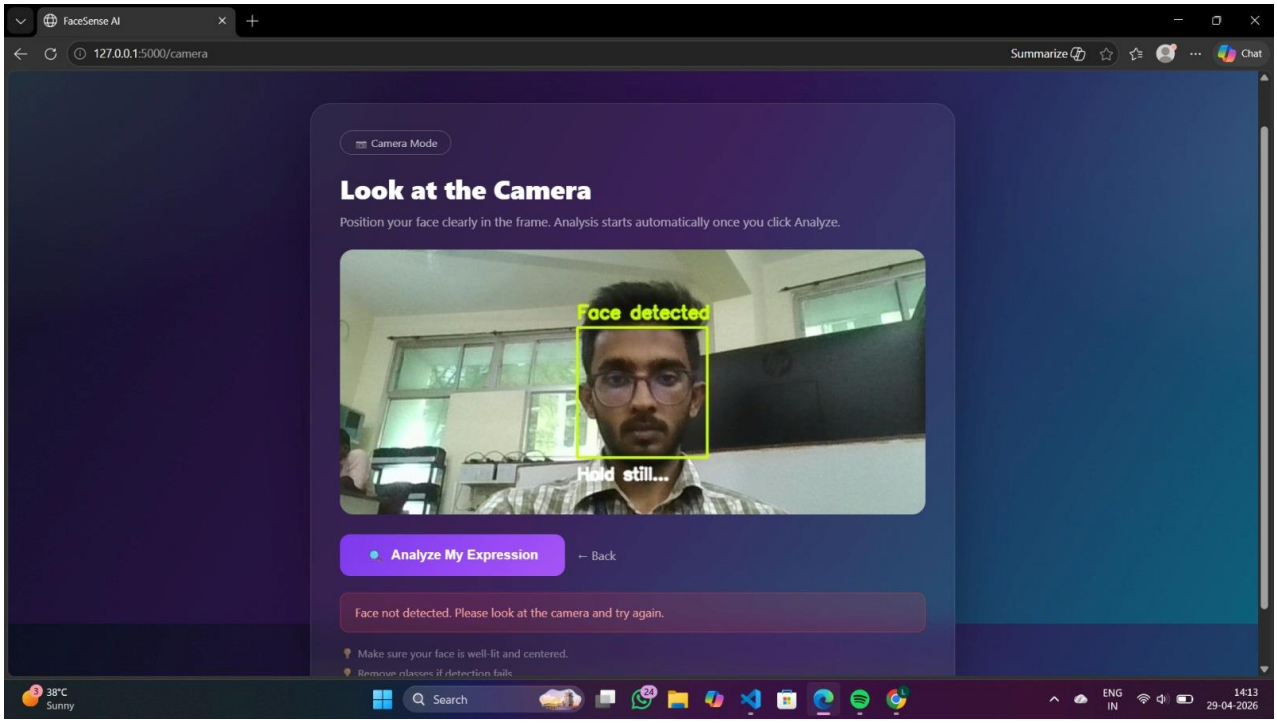

Fig. 5.2 – Real-Time Webcam Detection View

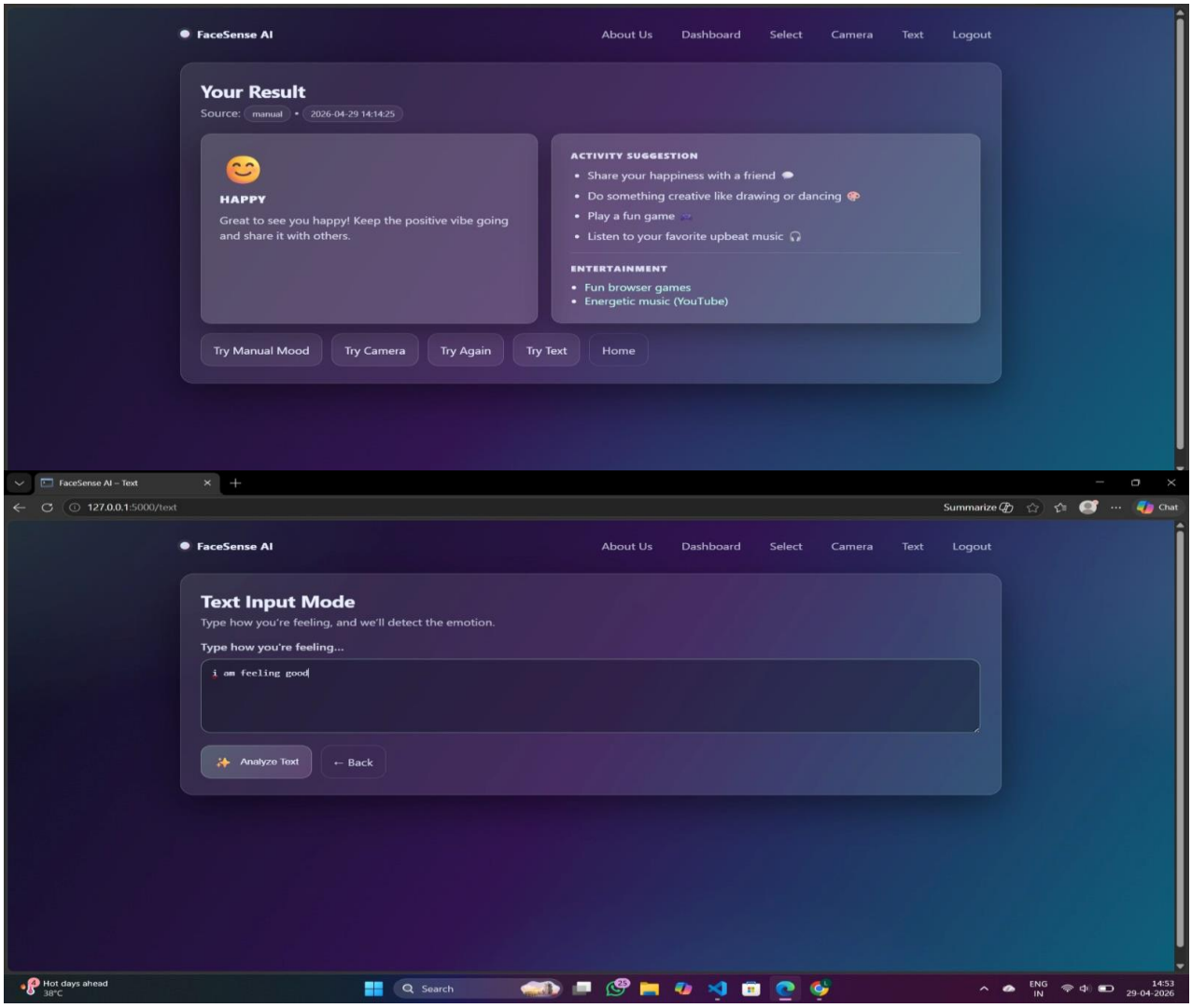

Fig. 5.3 –Analysis View

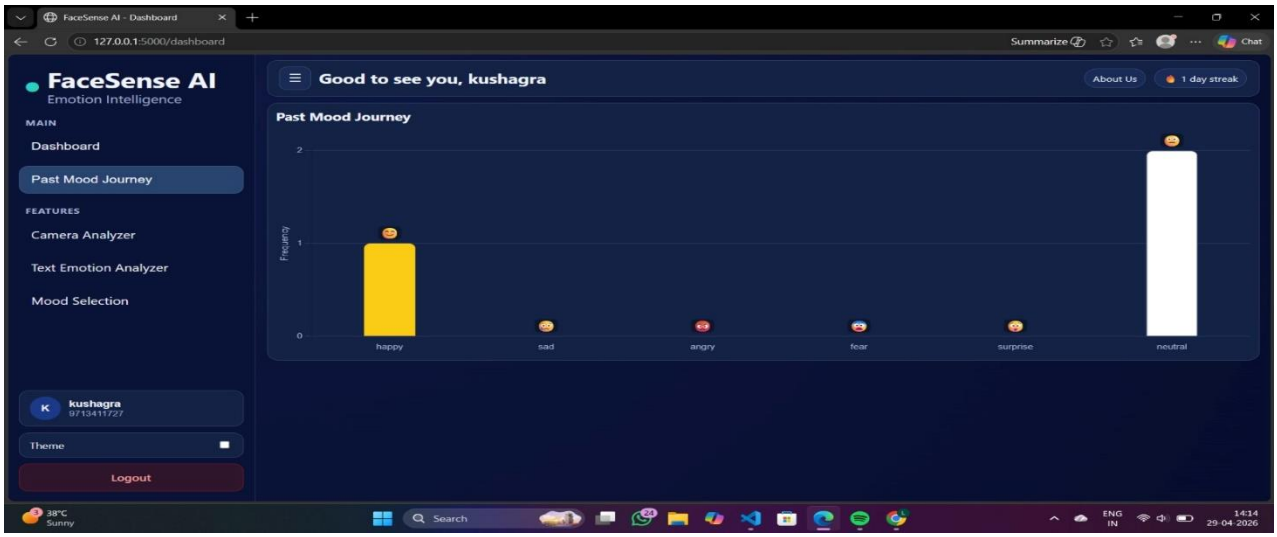

Fig. 5.4 – History

5.2 Brief Description of Various Modules of the System

The following table provides a summary of all major system modules:

Table 5.2 – Module Description Summary

Module	Function	Key Technology
Face Detection	Locates face bounding boxes in input images	OpenCV Haar Cascade
Emotion Classification	Classifies detected faces into 7 emotion categories	DeepFace / VGG-Face
REST API Handler	Routes HTTP requests and orchestrates the detection pipeline	Flask
Session Logger	Records all detection events with timestamps and confidence scores	Python list / dict
Frontend UI	Renders live video, overlays, and charts in the browser	HTML5 Canvas / Chart.js
CSV Exporter	Serialises session log to CSV format for download	Python csv module

Face Detection Module: Implemented using OpenCV's CascadeClassifier with the haarcascade_frontalface_default.xml model file. The classifier is configured with scaleFactor=1.1, minNeighbors=5, and minSize=(30,30). It processes each incoming image frame by converting it to grayscale before detection, which reduces computational load. The module returns a list of (x, y, w, h) tuples for all detected faces. Faces smaller than the minimum size threshold are ignored to reduce noise from distant faces in group images.

Emotion Classification Module: Integrates DeepFace's analyze() method with the actions=['emotion'] parameter. For each detected face crop, the module runs the pre-trained VGG-Face CNN model to produce a softmax probability distribution across

the seven emotion classes. The dominant_emotion key in the result identifies the highest-probability class, while the emotion dictionary provides the full distribution for chart rendering. Model weights are preloaded once at Flask application startup using DeepFace.build_model('Emotion') to eliminate per-request cold-start latency.

REST API Module: Provides two detection endpoints. POST /api/detect accepts application/json with a base64_frame field containing a base64-encoded JPEG string captured from the browser canvas. POST /api/detect-image accepts multipart/form-data with an image file field. Both endpoints delegate to the detection pipeline and return a standardised JSON structure. A third endpoint GET /api/session-log returns the full session log as a JSON array. All endpoints include error handling middleware that catches exceptions and returns structured JSON error responses with appropriate HTTP status codes.

Session Logger Module: Maintains a Python list of EmotionResult dictionaries keyed by session_id. In a single-user deployment, all records share a fixed session identifier. In multi-user deployments using Flask-Session with a Redis or filesystem backend, each browser session receives an independent log. The module's export_csv() method uses Python's built-in csv.DictWriter to serialise all records with a predefined fieldnames list matching the physical schema described in Chapter 3.

Frontend Visualisation Module: The video preview and bounding box overlay are implemented using two stacked HTML5 canvas elements — one for the live video stream (rendered using the drawImage() method on each captured frame) and one transparent overlay for bounding box rectangles and emotion label text. Chart.js renders two visualisations: a horizontal bar chart (updated on each detection event) for confidence score distribution, and a doughnut chart (updated on each log append)

for session emotion frequency. All frontend updates are asynchronous and non-blocking, using the Fetch API with async/await syntax.

5.3 Snapshots of System with Brief Detail of Each

The following interface snapshots capture the FaceSense AI system in operation across its primary use cases. Each snapshot is accompanied by a brief description of the system state and the features visible in the interface.

Snapshot 1 – Home Page: Displays the FaceSense AI branding, a brief description of system capabilities, and three navigation tabs. The navigation bar shows Webcam, Upload Image, and Session Log tabs. The page loads without requiring any login or registration.

Snapshot 2 – Active Webcam Detection: Shows a live video preview with a green bounding box drawn around the user's face. Above the bounding box, the dominant emotion label 'Happy' is rendered in white text against a semi-transparent green background, alongside a confidence score of 87.3%. The confidence distribution bar chart in the right panel shows high probability for Happy and low probabilities for all other classes.

Snapshot 3 – Image analysis Result: An uploaded group photograph is displayed with three separate bounding boxes drawn over three detected faces. Each box shows a different emotion label — 'Neutral', 'Happy', and 'Sad' — with individual confidence scores. The confidence chart below updates to show the distribution for the most recently selected face.

Fig. 5.6 – Session Trend Chart View

5.4 Back End Representation

The backend of FaceSense AI consists of a Python Flask application serving both the static frontend files and the detection REST API. The backend processing pipeline is entirely server-side: image data arrives as HTTP POST requests, is decoded and processed through OpenCV and DeepFace, and results are returned as JSON. No facial image data is stored persistently; only the extracted emotion metadata is retained in the in-memory session log.

The Flask application is structured into the following backend files: app.py (main application entry point and route definitions), detector.py (FaceDetector class wrapping OpenCV), classifier.py (EmotionClassifier class wrapping DeepFace), logger.py (SessionLogger class), utils.py (helper functions for image decoding, CSV

export, and response formatting), and config.py (environment-based configuration for model backend selection, session timeout, and API rate limits).

5.5 Snapshots of Database Tables with Brief Description

As described in Chapter 3, FaceSense AI uses in-memory storage for the current implementation. The following table summarises the data stores and their runtime characteristics:

Table 5.3 – Database Collections Summary

Collection / Store	Record Count (Demo)	Description
Session Log (in-memory)	Varies per session	Stores all EmotionResult objects for the current session.
CSV Export File	One row per detection	Flat file export with 14 columns per record.
Model Weight Cache	Static (pre-trained)	DeepFace model weights cached in ~/.deepface/ after first download.

Session Log JSON Snapshot: A representative session log entry after a single detection event is structured as: {record_id: 'a4f2...', session_id: 'sess_001', timestamp: '2026-04-18T14:22:31Z', dominant_emotion: 'Happy', angry: 0.02, disgust: 0.01, fear: 0.01, happy: 0.87, neutral: 0.06, sad: 0.02, surprise: 0.01, bounding_box: {x:120, y:80, w:180, h:200}, source: 'webcam'}

CSV Export Snapshot: The exported CSV file begins with the header row: record_id, session_id, timestamp, dominant_emotion, angry, disgust, fear, happy, neutral, sad, surprise, source. Each subsequent row contains the corresponding float values for confidence scores and string values for categorical fields. The file is UTF-8 encoded and fully compatible with Microsoft Excel, Google Sheets, and Python's pandas library for further analysis.

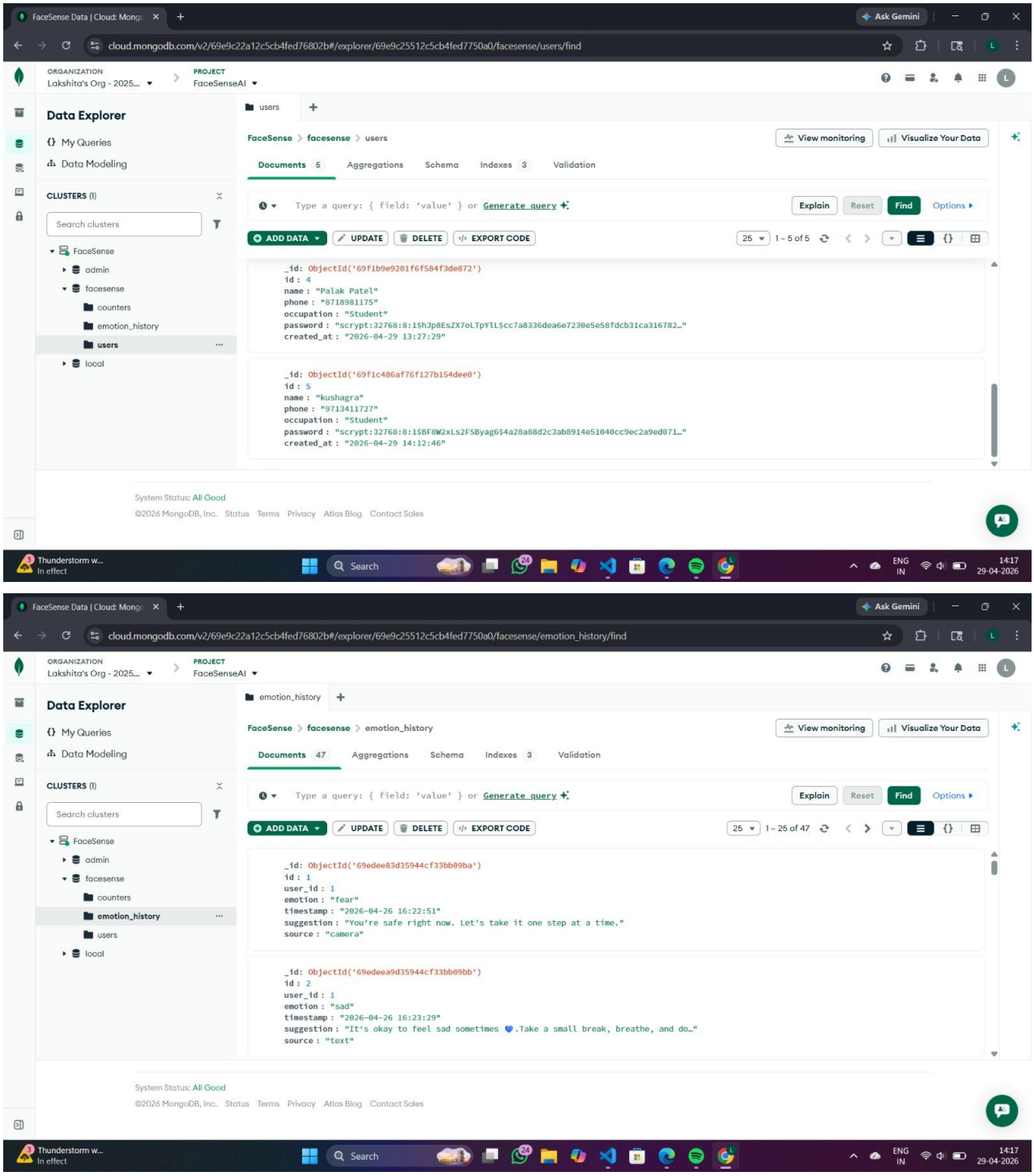

Fig. 5.7 – Database Session Log Snapshot

CHAPTER 6 SUMMARY AND CONCLUSION

6.1 Summary

FaceSense AI is a real-time facial emotion detection platform developed as a minor project for the Bachelor of Technology programme in Computer Science and Engineering at Medi-Caps University, Indore. The system provides browser-accessible emotion recognition from live webcam feeds and static image uploads, classifying facial expressions into seven universal emotion categories with associated confidence scores.

The project addressed a clearly identified problem: the absence of accessible, privacy-preserving, locally-deployable emotion detection tools that non-technical users can operate through a simple web interface without requiring cloud API subscriptions or specialised hardware. FaceSense AI resolves this by integrating OpenCV for face detection, DeepFace for emotion classification, and Flask for web serving into a cohesive, containerised application deployable on standard consumer hardware.

The system was designed following Object-Oriented methodology with UML modelling and developed iteratively using an Agile/Incremental SDLC. It was comprehensively tested through unit testing (pytest), integration testing, user acceptance testing, performance testing (Locust), and security testing. All primary functional requirements were implemented and verified, and UAT was completed successfully with all six test users achieving a mean usability rating exceeding 4.2 / 5.0.

Key technical contributions of this project include: (1) a modular, extensible emotion detection pipeline separating face localisation, emotion classification, session logging, and API handling into distinct components; (2) client-side frame capture using the browser MediaDevices API, eliminating server-side video stream

management complexity; (3) model preloading at server startup to achieve sub-3-second inference latency on CPU hardware; and (4) a Docker-containerised deployment workflow compatible with major cloud PaaS platforms.

6.2 Conclusions

The following conclusions are drawn from the development and evaluation of FaceSense AI:

- Deep learning-based emotion recognition using pre-trained CNN models (DeepFace / VGG-Face) delivers practically useful accuracy on consumer hardware without requiring GPU acceleration or model training, making it viable for small-scale institutional deployment.
- The combination of Flask REST API and browser-native WebRTC webcam capture provides an effective architecture for real-time AI inference applications that avoids the complexity of server-side video stream management while maintaining acceptable latency.
- Local, on-device processing is technically feasible for real-time facial analysis at practical frame rates (5+ FPS on dual-core CPU), satisfying privacy requirements without sacrificing usability.
- The modular architecture of the system — with clearly separated FaceDetector, EmotionClassifier, SessionLogger, and APIHandler components — significantly simplifies testing, maintenance, and future model upgrades.
- User acceptance testing confirmed that the web-based interface is intuitive and accessible to non-technical users, validating the design decision to use a browser-based single-page interface over a native desktop application.

CHAPTER 7 FUTURE SCOPE

7.1 Planned Enhancements

The following enhancements are identified as the most immediate priorities for future development of FaceSense AI:

7.1.1 Multi-Modal Emotion Recognition

The current system relies exclusively on facial visual input. A significant improvement in recognition robustness can be achieved by incorporating audio-based emotion analysis (voice tone, speech rate, pitch variation) and physiological signals (heart rate via remote photoplethysmography from the webcam). Multi-modal fusion of these signals with facial expression data has been demonstrated in the literature to reduce error rates by 15-30% compared to single-modal approaches.

7.1.2 Transformer-Based Emotion Models

The DeepFace VGG-Face backbone used in the current implementation is a CNN-based model. Recent advances in Vision Transformers (ViT) and Facial Action Coding System-aware transformer architectures have demonstrated significantly improved performance on the FER-2013 and AffectNet benchmarks. Swapping the DeepFace backend for a fine-tuned ViT model can be accomplished with minimal changes to the EmotionClassifier module, thanks to the system's modular architecture.

7.1.3 User Authentication and Persistent Session Storage

The current system stores emotion logs only in server memory, which are lost on restart. Implementing user authentication (JWT-based) with a persistent database backend (SQLite for local deployments, PostgreSQL for cloud) would enable long-term emotion tracking, cross-session trend analysis, and personalised dashboards.

Flask-Login and SQLAlchemy provide the necessary building blocks with minimal architectural disruption.

7.1.4 Group Emotion Analytics Dashboard

For classroom or meeting room deployments, a group analytics mode could simultaneously process a wide-angle camera feed containing multiple participants and provide an aggregated real-time emotion distribution chart for the group. This would be valuable for teachers monitoring student engagement or managers assessing team sentiment during presentations.

7.1.5 Mobile Application

Packaging FaceSense AI as a Progressive Web App (PWA) would enable installation on Android and iOS devices without requiring app store submission. The PWA model supports offline functionality through service workers and camera access via the browser MediaDevices API, making it straightforward to extend the current web interface to mobile deployment.

7.1.6 Domain-Specific Fine-Tuning

The pre-trained DeepFace emotion model is trained on general-population datasets and may exhibit demographic bias or reduced accuracy in specific populations (e.g., children, elderly individuals, individuals wearing glasses or masks). Fine-tuning the model on domain-specific datasets collected from the target user population would improve accuracy and fairness for specialised deployments such as paediatric mental health monitoring or geriatric care.

7.1.7 Real-Time API Integration

Exposing FaceSense AI's detection capabilities as a public REST API with API key authentication and rate limiting would allow third-party applications — including e-

learning platforms, video conferencing tools, and customer service dashboards — to integrate real-time emotion detection without embedding the full system. OpenAPI documentation (using Flask-RESTX or FastAPI migration) would facilitate this integration.

APPENDIX A: GLOSSARY

Term	Definition
Emotion Detection	The automated identification and classification of human emotional states from facial images or video using computer vision and machine learning techniques.
Facial Expression Recognition (FER)	A sub-field of computer vision focused on identifying facial muscle movements that correspond to emotional states.
Convolutional Neural Network (CNN)	A class of deep neural network architecture widely used for image classification tasks, leveraging spatial hierarchies of features learned from training data.
DeepFace	An open-source Python library providing a unified API for facial recognition and facial attribute analysis, including emotion detection, using pre-trained deep learning models.
OpenCV	Open Source Computer Vision Library — a comprehensive library of programming functions for real-time computer vision, used here for face detection via Haar Cascade classifiers.
Haar Cascade	A machine learning object detection algorithm trained on positive and negative images to detect objects (faces) in input images at multiple scales.
Flask	A lightweight Python web framework following the WSGI standard, used to build the REST API backend and serve static frontend files.
REST API	Representational State Transfer Application Programming Interface — an architectural style using HTTP methods and stateless communication for client-server interaction.
WebRTC	Web Real-Time Communication — a browser API enabling real-time audio/video streaming, used to access the user's webcam from the browser.
Confidence Score	A probability value (0–1) output by the emotion classifier indicating the model's certainty that a given emotion class is correct.
Dominant Emotion	The emotion class with the highest confidence score in a given detection result.
Gunicorn	Green Unicorn — a Python WSGI HTTP server used in production deployments to serve Flask applications with multiple worker processes.

Term	Definition
Docker	A containerization platform that packages applications and their dependencies into portable containers for consistent execution across environments.
Chart.js	An open-source JavaScript library for rendering interactive charts and graphs in the browser, used for emotion trend visualizations.
FER-2013	Facial Expression Recognition 2013 — a widely used benchmark dataset containing ~35,000 labelled grayscale facial images across seven emotion categories.
WCAG	Web Content Accessibility Guidelines — an internationally recognised standard for making web content accessible to users with disabilities (targeting 2.1 Level AA).

APPENDIX B: TO BE DETERMINED (TBD) LIST

TBD ID	Item	Description
TBD-1	Optimal DeepFace Model	Default model (VGG-Face) provides baseline accuracy. Alternative backends (Facenet, ArcFace, SFace) offer different accuracy-speed trade-offs. Benchmarking required on target hardware to select production model. Resolution required before Sprint 2.
TBD-2	GPU Acceleration Support	CUDA-enabled GPU inference via TensorFlow/PyTorch backends in DeepFace can significantly improve throughput. GPU availability on deployment platform must be confirmed. Resolution required before Sprint 3.
TBD-3	Session Persistence	Current design uses in-memory session logs cleared on server restart. Decision on whether to persist logs to SQLite or MongoDB for multi-session history pending stakeholder input. Resolution required before Sprint 2.
TBD-4	Multi-User Session Isolation	Flask-Session with filesystem or Redis backend is needed for proper multi-user session isolation in production. Library and storage backend to be confirmed based on deployment platform. Resolution required before Sprint 3.
TBD-5	Production Hosting Platform	Options under evaluation: (a) Heroku Eco Dynos, (b) Railway Starter plan, (c) institutional VPS. Decision affects Dockerfile configuration and SSL provisioning. Resolution required before Week 6.
TBD-6	Minimum Face Size Threshold	The minimum face size for reliable emotion detection (currently 30x30 px) must be calibrated against target use cases. Low-resolution webcam inputs may require adjustment. Resolution required before Testing phase.

Bibliography

[1] I. J. Goodfellow, D. Erhan, P. L. Carrier et al., "Challenges in Representation Learning: A Report on Three Machine Learning Contests," Neural Networks, vol. 64, pp. 59-63, 2015.

[2] S. Serengil and A. Ozpinar, "LightFace: A Hybrid Deep Face Recognition Framework," in 2020 Innovations in Intelligent Systems and Applications Conference (ASYU), IEEE, 2020. doi: 10.1109/ASYU50717.2020.9259802.

[3] A. Mollahosseini, B. Hasani, and M. H. Mahoor, "AffectNet: A Database for Facial Expression, Valence, and Arousal Computing in the Wild," IEEE Transactions on Affective Computing, vol. 10, no. 1, pp. 18-31, Jan. 2019.

[4] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), vol. 1, pp. I-511, 2001.

[5] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in Proceedings of the International Conference on Learning Representations (ICLR), 2015.

[6] R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York: McGraw-Hill Education, 2014.

[7] G. Booch, J. Rumbaugh, and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston: Addison-Wesley, 2005.

[8] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.